

```
"""
```

CARD RADAR - obchody 6.22 (nad vylepsenia 6.21)

- /admin/obchody?key=ADMIN_KEY : otestuješ ľubovoľný e-shop (aj z mobilu), CardRadar rozpozná platformu (Shoptet, Shopify, Upgates, WooCommerce), ukáže nájdené produkty a jedným ťuknutím obchod zapneš / vypneš. Zapnuté obchody sa ukladajú do databázy, netreba meniť kód ani deployovať. Všetky procesy servera si zmenu prevezmú do 30 sekúnd.
- ikony a logo bez metadát (cr-logo.png a ikony boli z polovice „prázdné“ dáta)
- viac vlákien na hľadanie, keď pribudnú obchody

INŠTALÁCIA - v app.py pod riadky

```
import vylepsenia
vylepsenia.install(globals())

pridaj:
import obchody
obchody.install(globals())

"""
```

```
import hmac
import html
import json
import struct
import time
import urllib.parse

import catalog_shops as cs

G = {}
```

Návrhy na otestovanie (NEOVERENÉ - stránka ich len ponúkne, nič sa nezapne samo
KANDIDATI = [

```
    ("Veselý drak", "https://www.vesely-drak.cz/"),
    ("Posbírej to", "https://www.posbirejto.cz/"),
    ("Najáda", "https://www.najada.games/"),
    ("Tlama Games", "https://www.tlamagames.com/"),
    ("Blackfire", "https://www.blackfire.cz/"),
    ("Xzone CZ", "https://www.xzone.cz/"),
    ("Xzone SK", "https://www.xzone.sk/"),
    ("Gengar.cz", "https://www.gengar.cz/"),
]
```

```
META_KEY = "extra_shops"
_state = {"v": None, "t": 0.0}
```

```
# ----- databáza -----
```

```
def _meta_get(k):
    conn = cs._db()
    try:
        row = conn.execute("SELECT v FROM meta WHERE k = ?", (k,)).fetchone()
        return row[0] if row else None
    finally:
        conn.close()
```

```
def _meta_set(k, v):
    conn = cs._db()
    try:
        conn.execute("INSERT OR REPLACE INTO meta (k, v) VALUES (?, ?)", (k, v))
        conn.commit()
    finally:
        conn.close()
```

```
def _load():
    try:
        return json.loads(_meta_get(META_KEY) or "[]")
    except Exception:
        return []
```

```
def _save(shops):
    _meta_set(META_KEY, json.dumps(shops, ensure_ascii=False))
    _meta_set(META_KEY + "_v", str(time.time()))
    _state["t"] = 0 # tento proces sa zosynchronizuje hneď
```

```
# ----- zapnutie obchodov v bežiacom serveri -----
```

```
def _apply(configs):
    g = G["_g"]
    want = {c["name"] for c in configs}
    for lst in (g["SHOPS"], g["ACTIVE_SHOPS"]):
        lst[:] = [s for s in lst if not s.get("_extra") or s["name"] in want]
    have = {s["name"] for s in g["SHOPS"]}
    for c in configs:
        if c["name"] in have:
            continue
        shop = dict(c, enabled=True, _extra=True)
```

```

        g["SHOPS"].append(shop)
        g["ACTIVE_SHOPS"].append(shop)
        g["ALLOWED_HOSTS"].add(urllib.parse.urlparse(shop["base_url"]).netloc.lower)
# viac obchodov = viac súbežných vlákien (ThreadPoolExecutor ich tvorí podľa
ex = g["SHOP_EXECUTOR"]
ex._max_workers = max(ex._max_workers, len(g["ACTIVE_SHOPS"]) * 3)

def _sync():
    if time.monotonic() - _state["t"] < 30:
        return
    _state["t"] = time.monotonic()
    try:
        v = _meta_get(META_KEY + "_v")
        if v != _state["v"]:
            _state["v"] = v
            _apply(_load())
    except Exception:
        pass

# ----- rozpoznanie obchodu -----

def detect(url, q):
    g = G["_g"]
    url = url.strip()
    if "://" not in url:
        url = "https://" + url
    p = urllib.parse.urlparse(url)
    if p.scheme not in ("http", "https") or not p.netloc:
        return {"error": "Neplatná adresa."}
    base = f"{p.scheme}://{p.netloc}/"
    resp, dbg = g["fetch"](base, timeout=10)
    if not resp:
        return {"error": f"Stránka neodpovedá ({dbg.get('error') or dbg.get('stat
platform = next((n for n, pr in g["PLATFORM_PRESETS"].items()
                if pr["marker"].search(resp.text)), None)
    if not platform:
        return {"error": "Platformu sa nepodarilo rozpoznať. Takýto obchod pôjde
                "alebo ručné nastavenie."}
    host = p.netloc.lower()
    preset = g["PLATFORM_PRESETS"][platform]
    norm = g["normalize_query"](q).get("normalized") or q
    tried = []
    for path in preset["search"]:
        shop = {"name": host.replace("www.", ""), "country": "CZ" if host.endswit
                "base_url": base, "search_url": base.rstrip("/") + path,

```

```

        "link_selector": preset["selector"]}]
if platform == "shopify":
    shop["shopify"] = True
try:
    res, d = g["_scrape"](shop, norm, 10)
except Exception as e:
    res, d = [], {"status": "chyba", "error": str(e)[:120]}
    tried.append({"url": shop["search_url"], "status": d.get("status"),
                  "links": d.get("links_scanned", 0), "results": len(res)})
if res:
    return {"platform": platform, "config": shop, "sample": res[:8], "tri
return {"platform": platform, "tried": tried,
        "error": "Platforma rozpoznaná, ale vyhľadávanie nevrátilo produkty."

# ----- admin stránka -----

def _key_ok(key):
    real = G["_g"] ["ADMIN_KEY"]
    return bool(real) and hmac.compare_digest((key or "").encode(), real.encode())

def _page(body):
    from flask import Response
    return Response(f'<!doctype html><html lang="sk"><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1"><title>Obchody
<style>{cs._PAGE_CSS}
input,button{{font:inherit;padding:10px;border-radius:9px;border:1px solid #d3d9e
button{{background:#ffcf3a;border:0;font-weight:800}} .row{{display:flex;gap:8px;
.ok{{color:#0a8a4a}} .err{{color:#d4334b}} li{{margin:4px 0}} small{{color:#8b94a
<header><a href="/">CardRadar</a></header><main>{body}</main>""', mimetype="text/
        headers={"Cache-Control": "no-store"})

def admin_obchody():
    from flask import request, redirect
    e = html.escape
    key = request.values.get("key", "")
    if not _key_ok(key):
        return _page("<h1>Nepovolené</h1><p>Pridaj ?key=ADMIN_KEY</p>"), 403
    _sync()
    saved = _load()
    base = "/admin/obchody?key=" + urllib.parse.quote(key)

    if request.method == "POST":
        act = request.form.get("act")
        if act == "add":

```

```

try:
    c = json.loads(request.form.get("config", ""))
    ok = (urllib.parse.urlparse(c["search_url"]).netloc ==
          urllib.parse.urlparse(c["base_url"]).netloc)
except Exception:
    ok = False
if ok:
    c["name"] = (request.form.get("name") or c["name"]).strip()[:40]
    c = {k: c[k] for k in ("name", "country", "base_url", "search_url",
                          "link_selector", "shopify") if k in c}
    saved = [s for s in saved if s["name"] != c["name"]] + [c]
    _save(saved)
elif act == "remove":
    _save([s for s in saved if s["name"] != request.form.get("name")])
_sync()
return redirect(base, code=303)

h = "<h1>Obchody</h1><h2>Pridané cez túto stránku</h2>"
if saved:
    h += "<ul>" + "".join(
        f"<li><b>{e(s['name'])}</b> <small>{e(s['country'])} · {e(s['search_u"
        f"<form method=post class=row><input type=hidden name=key value='{e(k"
        f"<input type=hidden name=act value=remove><input type=hidden name=na"
        f"<button>Vypnúť</button></form></li>" for s in saved) + "</ul>"
else:
    h += "<p>Zatiaľ žiadne.</p>"

test = request.args.get("test", "").strip()
q = request.args.get("q", "").strip() or "pikachu"
h += (f"<h2>Otestovať obchod</h2><form class=row><input type=hidden name=key"
      f"<input name=test placeholder='https://www.obchod.cz' value='{e(test)}'"
      f"<input name=q value='{e(q)}' style='width:130px'><button>Test</button"

if test:
    r = detect(test, q)
    if r.get("platform"):
        h += f"<p>Platforma: <b>{e(r['platform'])}</b></p>"
    for t in r.get("tried", []):
        h += f"<p><small>{e(t['url'])} → {e(str(t['status']))}, odkazov {t['l"
    if r.get("error"):
        h += f"<p class=err>{e(r['error'])}</p>"
    if r.get("config"):
        h += "<p class=ok>Funguje. Ukážka:</p><ul>" + "".join(
            f"<li>{e(x['title'])} - <b>{x['price_eur']:.2f} €</b> <small>{e(x"
            for x in r["sample"]) + "</ul>"
        h += (f"<form method=post class=row><input type=hidden name=key value"
              f"<input type=hidden name=act value=add>"

```

```

        f"<input type=hidden name=config value='{e(json.dumps(r['config'
        f"<input name=name value='{e(r['config']['name'])}'>"
        f"<button>Zapnúť obchod</button></form>"
        f"<p><small>Skontroluj ceny v ukážke. Ak sedia, zapni.</small><

h += "<h2>Návrhy na otestovanie</h2><ul>" + "".join(
    f"<li><a href='{e(base)}&test={urllib.parse.quote(u)}'>{e(n)}</a> <small>
    for n, u in KANDIDATI) + "</ul>"
return _page(h)

# ----- menšie obrázky -----

_DROP = {b"caBX", b"tEXt", b"iTXt", b"zTXt", b"eXIf"}

def strip_png(data):
    """PNG bez metadát (C2PA, texty). Obrázok ostáva rovnaký."""
    if data[:8] != b"\x89PNG\r\n\x1a\n":
        return data
    out, i = [data[:8]], 8
    try:
        while i < len(data):
            length = struct.unpack(">I", data[i:i + 4])[0]
            ctype = data[i + 4:i + 8]
            chunk = data[i:i + 12 + length]
            if ctype not in _DROP:
                out.append(chunk)
            i += 12 + length
            if ctype == b"IEND":
                break
    except Exception:
        return data
    return b"".join(out)

def install(g):
    G.update(g)
    G["_g"] = g
    for name, (data, mime) in list(g["EMBEDDED_STATIC"].items()):
        if mime == "image/png":
            g["EMBEDDED_STATIC"][name] = (strip_png(data), mime)
    app = g["app"]
    app.add_url_rule("/admin/obchody", "admin_obchody", admin_obchody, methods=["
    app.before_request(lambda: _sync() and None)
    _sync()

```

